System Design Report

Vehicle Monitoring System (VMS) — Distributed System Using SocketCAN

Name: Sohan Satish Sutar BE170 | Course: Automotive Communication Networks (KPIT APEX Lab) | Problem Statement 2 Mini Project

1. Problem Description

1.1 Selected system and scope

The assignment asks for a distributed system of software ECUs that communicate over SocketCAN, with diagnostics, fault handling and verification. The selected application is a **Vehicle Monitoring System (VMS)**: it continuously observes vehicle speed, engine speed, engine-coolant temperature, battery voltage and fuel level, decides on warnings and cooling, drives a fan, warning lamps and a buzzer, and records faults. A vehicle monitoring system is not one of the five listed options; it is therefore proposed as **Option 6 (student-proposed system)** and must be approved by the instructor before implementation. Its functions overlap Option 3 (vehicle diagnostics) and Option 5 (voltage/temperature monitoring and fault detection), which keeps the scope within what the assignment expects.

1.2 Objectives

- Acquire five vehicle signals periodically and distribute them on a CAN bus (500 kbit/s, 11-bit identifiers).
- Evaluate the signals against plausibility limits and warning thresholds; select a system mode (NORMAL, DEGRADED, SAFE).
- Drive a cooling fan (OFF/LOW/HIGH), warning lamps and a buzzer, with a defined safe state when information is missing.
- Detect missing messages, invalid values and failed ECUs; report them as diagnostic trouble codes (DTCs) with a timestamped event log and a console dashboard.

1.3 Assumptions and constraints

The system runs on Linux with a virtual SocketCAN interface (vcan0); each ECU is a separate Python process using raw AF_CAN sockets from the standard library. Sensor values are deterministic synthetic profiles (speed ramps to 90 km/h in 30 s, coolant rises from 60 °C to 92 °C, battery about 13.8 V, fuel decreasing slowly), which makes every test repeatable. Classical CAN is used; no authentication or encryption is implemented.

2. Functional Architecture (Task 1)

Table 1 lists the functional analysis; Figure 1 shows the functional block diagram.

Table 1: Functional analysis of the VMS

Item	Description
System objectives	Monitor vehicle health, protect the engine from overheating, warn the driver, remain safe under faults, and expose diagnostics.
Inputs	Vehicle speed, engine speed, coolant temperature, battery voltage, fuel level; test commands (fault injection) from the technician.
Outputs	Cooling-fan command, lamp mask (temperature, battery, fuel, general), buzzer, alarm events, DTC reports, dashboard.
Sensors (simulated)	Wheel/vehicle speed sensor, crankshaft speed sensor, coolant NTC sensor, battery voltage sense, fuel-level sender.
Actuators (simulated)	Radiator cooling fan (3 states), instrument warning lamps (4 bits), audible buzzer.
User interactions	Driver/technician reads the console dashboard; technician injects faults with keys 0–5 and stops/restarts ECU processes.

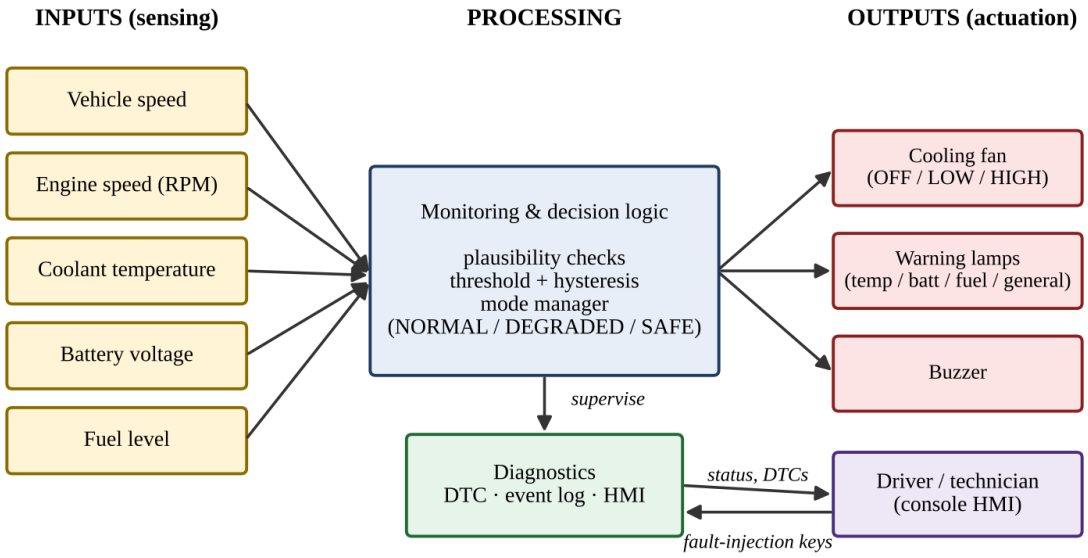


Figure 1: Functional block diagram of the Vehicle Monitoring System

3. ECU Architecture (Task 2)

The system is divided into four independent ECUs, each implemented as its own application and started as its own operating-system process. No ECU shares memory with another; all information moves as CAN frames (Figure 2, Table 2).

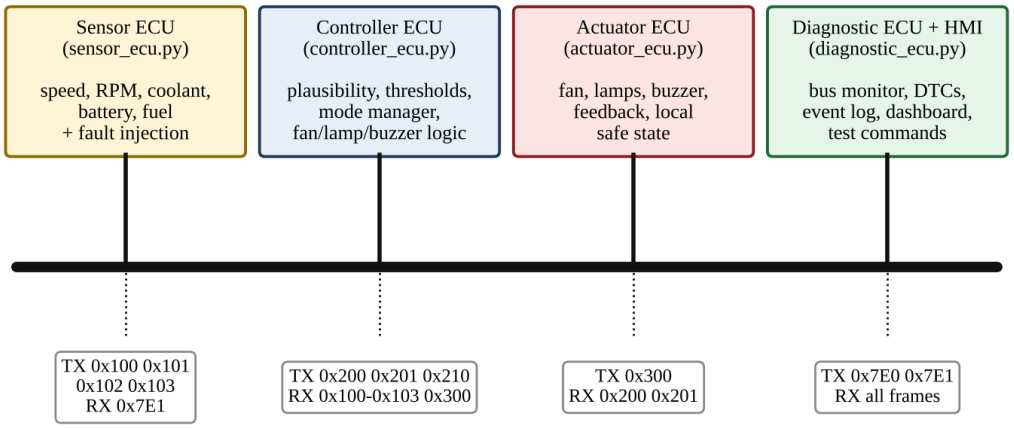


Figure 2: ECU architecture on the SocketCAN bus

Table 2: ECU responsibilities

ECU / process	Responsibility	Transmits	Receives
Sensor ECU – sensor_ecu.py	Generates and transmits the five signals with an alive counter; accepts test commands (inject invalid value, mute transmission).	0x100–0x103	0x7E1
Controller ECU – controller_ecu.py	Supervises sensor and actuator messages, checks ranges and thresholds (with hysteresis), manages the mode, commands fan/lamps/buzzer, raises alarm events.	0x200, 0x201, 0x210	0x100–0x103, 0x300
Actuator ECU – actuator_ecu.py	Applies fan/lamp/buzzer commands, reports the applied state, enters a local safe state if commands stop.	0x300	0x200, 0x201
Diagnostic ECU + HMI – diagnostic_ecu.py	Listens to all traffic; detects missing messages and abnormal values independently of the controller; keeps DTCs and the event log; shows the dashboard; sends fault-injection commands.	0x7E0, 0x7E1	All (0x100–0x300)

4. CAN Network Design (Task 3)

4.1 CAN matrix

Ten messages are defined (requirement: at least eight, with at least two periodic and two event-triggered). Seven are periodic (0x200 additionally transmits immediately when a command changes) and three are event-triggered.

Table 3: CAN matrix (500 kbit/s, standard 11-bit identifiers)

CAN ID	Message	Transmitter	Receivers	Type	Rate	DLC
0x100	MOTION_DATA	Sensor	Controller, Diagnostic	Periodic	20 ms	6
0x101	COOLANT_TEMP	Sensor	Controller, Diagnostic	Periodic	100 ms	2

CAN ID	Message	Transmitter	Receivers	Type	Rate	DLC
0x102	BATTERY_VOLT	Sensor	Controller, Diagnostic	Periodic	100 ms	2
0x103	FUEL_LEVEL	Sensor	Controller, Diagnostic	Periodic	500 ms	1
0x200	CTRL_COMMAND	Controller	Actuator, Diagnostic	Periodic + on change	50 ms	4
0x201	ALARM_EVENT	Controller	Actuator, Diagnostic	Event	on event	3
0x210	CTRL_STATUS	Controller	Diagnostic	Periodic	100 ms	4
0x300	ACTUATOR_STATUS	Actuator	Controller, Diagnostic	Periodic	100 ms	5
0x7E0	DIAG_FAULT_RPT	Diagnostic	All (HMI/log)	Event	on event	4
0x7E1	TEST_COMMAND	Diagnostic	Sensor	Event	on event	2

4.2 Identifier allocation and arbitration

Lower identifiers win arbitration. Identifiers are grouped by function: 0x1xx sensor data, 0x2xx controller commands and events, 0x3xx actuator feedback, 0x7xx diagnostics. The fastest and most safety-relevant data (0x100) therefore has the highest priority, and diagnostic traffic can never delay control traffic. Each message has exactly one transmitter (message ownership), so no identifier conflicts exist.

4.3 Bus load

The worst-case length of a standard frame with n data bytes including stuff bits and interframe space is $47 + 8n + \lfloor (34 + 8n - 1)/4 \rfloor$ bits. Summing the seven periodic messages at their rates gives **11 280 bit/s = 2.26 % of 500 kbit/s** (1.87 % without stuffing). The bus therefore has large reserve for event frames and for future signals. Per-message figures are given in the Test Report.

5. Signal Definitions (Task 4)

All multi-byte signals use Intel (little-endian) byte order. Physical value = raw × resolution. Encoding saturates at the raw limits instead of wrapping, so an out-of-range input cannot alias into a plausible value. The maximum quantisation error is half a resolution step: ±0.005 km/h, ±0.125 rpm, ±0.05 °C, ±0.005 V, ±0.25 %.

Table 4: Signal specification

Signal	Message · bytes	Type	Resolution	Valid range / notes
Vehicle speed	0x100 · B0–B1	uint16	0.01 km/h	0–300 km/h (raw 0–30 000)
Engine speed	0x100 · B2–B3	uint16	0.25 rpm	0–8000 rpm (raw 0–32 000)
Alive counter	0x100 · B4	uint8	1	Rolling 0–255; a gap = lost frame
Sensor status	0x100 · B5	uint8	—	Reserved (0)
Coolant temperature	0x101 · B0–B1	int16	0.1 °C	−40 to 150 °C (raw −400 to 1500)
Battery voltage	0x102 · B0–B1	uint16	0.01 V	6.00–18.00 V (raw 600–1800)
Fuel level	0x103 · B0	uint8	0.5 %	0–100 % (raw 0–200; >200 invalid)
Fan command	0x200 · B0	enum	—	0 OFF, 1 LOW, 2 HIGH

Signal	Message · bytes	Type	Resolution	Valid range / notes
Lamp mask	0x200 · B1	bit field	—	b0 temp, b1 battery, b2 fuel, b3 general
Buzzer; counter	0x200 · B2; B3	uint8	—	Buzzer 0/1; rolling counter
Alarm code, severity, state	0x201 · B0, B1, B2	uint8 × 3	—	Code 0x01–0x20; severity 1–3; 1 = set, 0 = cleared
Controller mode	0x210 · B0	enum	—	0 NORMAL, 1 DEGRADED, 2 SAFE
Alarm mask; counter	0x210 · B1–B2; B3	uint16; uint8	—	One bit per alarm; rolling counter
Actuator state	0x300 · B0–B4	uint8 × 5	—	Fan, lamps, buzzer, acknowledged counter, local-safe flag
DTC report	0x7E0 · B0–B3	uint16 + 2 × uint8	—	DTC, source ECU (1–3), status (1 active, 0 cleared)
Test command	0x7E1 · B0–B1	uint8 × 2	—	Target 1 = Sensor; command 0 clear, 1–5 inject, 3 mute

6. Software and Fault-Handling Design

6.1 Software structure

The ECU behaviour is written once, in hardware-independent classes (ecus.py) that only need three operations: send(id, data), on_frame(time, id, data) and tick(time). Two back-ends drive them: a SocketCAN runner for real operation (one process per ECU) and a virtual-time bus model used to compute the observations in the Test Report. The same logic is therefore exercised in both.

6.2 Supervision and detection

Every periodic message is supervised with a timeout of **three times its period** (tolerating two lost frames). The Diagnostic ECU checks all timeouts every 10 ms, so the worst-case detection time is the timeout plus 10 ms plus one frame time, i.e. 70 ms for 0x100, 310 ms for 0x101/0x102 and 1510 ms for 0x103. The Controller evaluates its inputs every 10 ms and sends 0x200 immediately when a command changes, so the reaction to a fault is about one evaluation period. The alive counter in 0x100 lets the Diagnostic ECU count lost frames even without a timeout.

6.3 Fault-handling matrix

Table 5: Detection rules and system responses

Condition	Detected by	System response
Sensor message missing (> 3 × period)	Controller, Diagnostic	SAFE mode: fan HIGH, temperature + general lamp, buzzer. DTC 0xC100–0xC103.
Coolant outside −40…150 °C	Controller, Diagnostic	SAFE mode (cannot trust temperature). DTC 0xB101.
Battery outside 6–18 V	Controller, Diagnostic	DEGRADED mode, battery + general lamp. DTC 0xB102.
Coolant ≥ 105 °C (clear < 100 °C)	Controller	DEGRADED, fan HIGH, temperature lamp.
Coolant ≥ 115 °C (clear < 110 °C)	Controller	SAFE, fan HIGH, buzzer.

Condition	Detected by	System response
Actuator status 0x300 missing > 300 ms	Controller, Diagnostic	DEGRADED. DTC 0xC300. Other ECUs keep running.
Controller command 0x200 missing > 150 ms	Actuator, Diagnostic	Actuator local safe state: fan HIGH, lamp, buzzer. DTC 0xC200, 0xC210.
Battery < 11.5 V or > 15.0 V; fuel ≤ 10 %	Controller	Warning lamp only (hysteresis 0.3 V / 2 %).

DTC numbering: 0xC000 + message ID for a missing message (e.g. 0xC300 = 0x300 missing) and 0xB1xx for an out-of-range signal. A fault clears automatically when the message returns or the value is plausible again, and each set/clear is sent as an event frame (0x7E0) and written to the log, for example: "WARNING – Sensor ECU timeout: message 0x101 missing > 300 ms; action: controller expected to enter SAFE mode".

6.4 HMI

The Diagnostic ECU renders a console dashboard every 0.5 s showing speed, RPM, coolant, battery, fuel, controller mode, actuator states, active DTCs and the five most recent events. Keys 1–5 inject faults, 3 mutes the Sensor ECU and 0 clears injected faults.

7. Design Decisions

- **Fail-safe cooling.** When sensor information is lost or implausible the fan goes to HIGH and the buzzer sounds, because assuming the worst case protects the engine; the cost is noise and energy.
- **Timeout = 3 × period.** Tolerates two consecutive lost frames without false alarms while keeping detection fast (70 ms for the fastest signal).
- **Hysteresis on every threshold.** Prevents fan and lamp chatter near a limit (e.g. warning set at 105 °C, cleared below 100 °C).
- **Independent diagnostics.** The Diagnostic ECU re-checks ranges and timeouts itself, so a fault in the Controller's own logic can still be detected.
- **Fault injection over CAN (0x7E1).** Test faults travel over the bus like real traffic, so the whole chain is exercised and no back door into the ECUs is needed.
- **Periodic + on-change command.** 0x200 repeats every 50 ms (so loss is detectable) but also sends immediately on change (so reaction is fast).
- **Limitations.** Periodic frames start in phase, so they queue together after start-up (a start offset per message would reduce jitter); no message authentication; timestamps are relative to process start.

8. Project Structure and Execution

File	Purpose
can_matrix.py	Message IDs, CAN matrix, encode/decode, limits
ecus.py	ECU logic (Sensor, Controller, Actuator, Diagnostic)
socketcan_io.py	SocketCAN back-end and process runner

File	Purpose
sensor_ecu.py, controller_ecu.py, actuator_ecu.py, diagnostic_ecu.py	One entry point (process) per ECU
sim_bus.py, analyse.py, test_vms.py	Virtual-time bus model, observation extraction, 11 automated tests
setup_vcan.sh, run_all.sh	Create vcan0 and start the system

Run (Linux):

```
sudo ./setup_vcan.sh vcan0
./run_all.sh vcan0          # Diagnostic ECU / HMI in foreground
candump -td vcan0           # second terminal: live CAN traffic
```